

RTOS-Based Mixed-Criticality Smart Vehicle Safety System

with Adaptive Task Management

Sr.	Name	Roll No.	PRN	Batch
1	Avishkar Gadhave	312011	22310286	B2
2	Mahesh Kachave	312015	22310389	B2
3	Vinayak Desai	312070	22420144	B2

1. Abstract

This report presents the design, implementation, and validation of an RTOS-Based Mixed-Criticality Smart Vehicle Safety System on the STM32F4 Nucleo microcontroller using FreeRTOS (CMSIS-RTOS v2). The system manages vehicle safety functions — obstacle detection via IR sensor, motor/buzzer control, CPU load monitoring, and dynamic stress injection — in a priority-driven, deterministic manner.

The key innovation is adaptive task management: when CPU load exceeds 80%, the system autonomously enters Critical Mode, suspending non-essential tasks (logging, LED indication) to guarantee responsiveness of safety-critical paths (IR sensing, braking). This Mixed-Criticality approach mirrors industrial embedded design patterns used in automotive, aerospace, and industrial control domains.

2. Introduction & Objectives

Modern embedded vehicle systems must handle tasks of widely varying criticality — from life-critical brake actuation to non-critical telemetry — on a single constrained processor. A naive superloop or equal-priority approach cannot guarantee safety-critical timing under varying load. This project addresses that gap through a CPU-aware, adaptive RTOS task management system.

Key objectives:

- Implement a multi-task RTOS system on STM32 for obstacle detection and vehicle safety.
- Demonstrate priority-based preemptive scheduling with FreeRTOS.
- Develop a CPU load monitoring subsystem using the DWT cycle counter.
- Implement adaptive task management — suspending non-critical tasks under high CPU load.
- Validate via UART serial output and hardware actuator response, and simulate overload with a stress task.

3. System Architecture

3.1 Hardware Platform

The system is built on the STM32F4 Nucleo (ARM Cortex-M4, FPU, up to 84 MHz, HSI oscillator). Key peripherals:

Pin / Peripheral	GPIO / Config	Function
Dc_motor_Pin (PA)	Output PP	DC motor — HIGH = running, LOW = stopped
Buzzer_Pin (PB)	Output PP	Buzzer — HIGH = alarm active
IRF_Pin	Input (No Pull)	IR sensor — LOW = obstacle detected
Button_Pin (PC13)	EXTI Falling	User button — triggers stress task injection

USART2	115200, 8N1	UART debug terminal via printf redirect
DWT CYCCNT	CoreDebug/DWT	Cycle counter for CPU load measurement

3.2 Software Stack

Three-layer architecture: (1) HAL Layer — STM32 Hardware Abstraction Library for GPIO, UART, timers and interrupts. (2) RTOS Layer — FreeRTOS v10 via CMSIS-RTOS v2 API (thread management, mutexes, event flags). (3) Application Layer — six user-defined tasks implementing vehicle safety logic.

4. Task Design & Inter-Task Communication

4.1 Task Inventory

Six tasks are created at startup. Priorities follow Rate Monotonic Scheduling (RMS) — higher-criticality, higher-frequency tasks get higher priorities.

Task	Priority	Period	Responsibility
BreakTask	High	50 ms	Motor OFF + Buzzer ON on obstacle; reverse otherwise
IRSensorTask	AboveNormal	50 ms	Reads IR pin; sets/clears OBSTACLE_BIT event flag
MonitorTask	Normal2	1000 ms	CPU load calculation, critical mode management, UART status
LoggingTask	Low7	1000 ms	Periodic UART LOG message; first suspended in critical mode

4.2 Key Task Details

IRSensorTask polls the IR pin every 50 ms. On LOW (obstacle), it sets OBSTACLE_BIT via `osEventFlagsSet()`; on HIGH (clear) it clears the bit. BreakTask (highest priority) reads OBSTACLE_BIT non-blocking and drives motor/buzzer GPIOs — guaranteeing worst-case brake response of 50 ms.

MonitorTask implements software CPU idle measurement using the FreeRTOS Idle Hook and DWT counter. A 2-second calibration baseline is captured at startup; subsequent 1-second windows compute: $\text{CPU\%} = 100 - (\text{current_idle} / \text{max_idle}) \times 100$. When load exceeds 80%, Critical Mode is triggered permanently.

4.3 Adaptive Critical Mode (Mixed-Criticality)

When CPU load > 80%, MonitorTask transitions to Critical Mode — the system sheds low-criticality load to protect high-criticality deadlines:

- LoggingTask suspended via `osThreadSuspend()` — halting all LOG UART output.
- MonitorTask switches from UART status printing to LED toggling as status indicator.
- BreakTask and IRSensorTask continue uninterrupted at their 50 ms deadlines.

4.4 IPC: Event Flags & Mutex

A single event flags object (`systemEventsHandle`) communicates obstacle state across tasks — appropriate since multiple tasks read binary state and event flags are ISR-safe. A `uartMutex` serializes all `printf()` calls across tasks using `osMutexAcquire / Release` with `osWaitForever`, preventing interleaved/garbled UART output.

The user button (PC13 EXTI) sets a volatile flag `stress_request = 1` in its ISR callback. `MonitorTask` polls this flag and dynamically spawns `StressTask` at runtime via `osThreadNew()` — demonstrating runtime task creation unavailable in bare-metal designs.

5. Stress Testing & Results

5.1 StressTask

`StressTask` consumes maximum CPU cycles via a floating-point loop (500,000 FPU multiplications per iteration, 1 ms yield between iterations). The volatile qualifier prevents compiler optimization. This saturates available cycles while still yielding for higher-priority task preemption.

5.2 Observed Results

State	Terminal Output	System Behavior
Normal — No Obstacle	LOG / System Running No Obstacle CPU = 50%	Motor ON, Buzzer OFF, all tasks active
Normal — Obstacle	System Running Obstacle Detected CPU = 50%	Motor OFF, Buzzer ON, IR flag set
Critical Mode — Stress	===== CRITICAL MODE ===== CPU = 99%	LoggingTask suspended, LED toggles, safety tasks continue at 50 ms

5.3 Terminal Output Screenshots

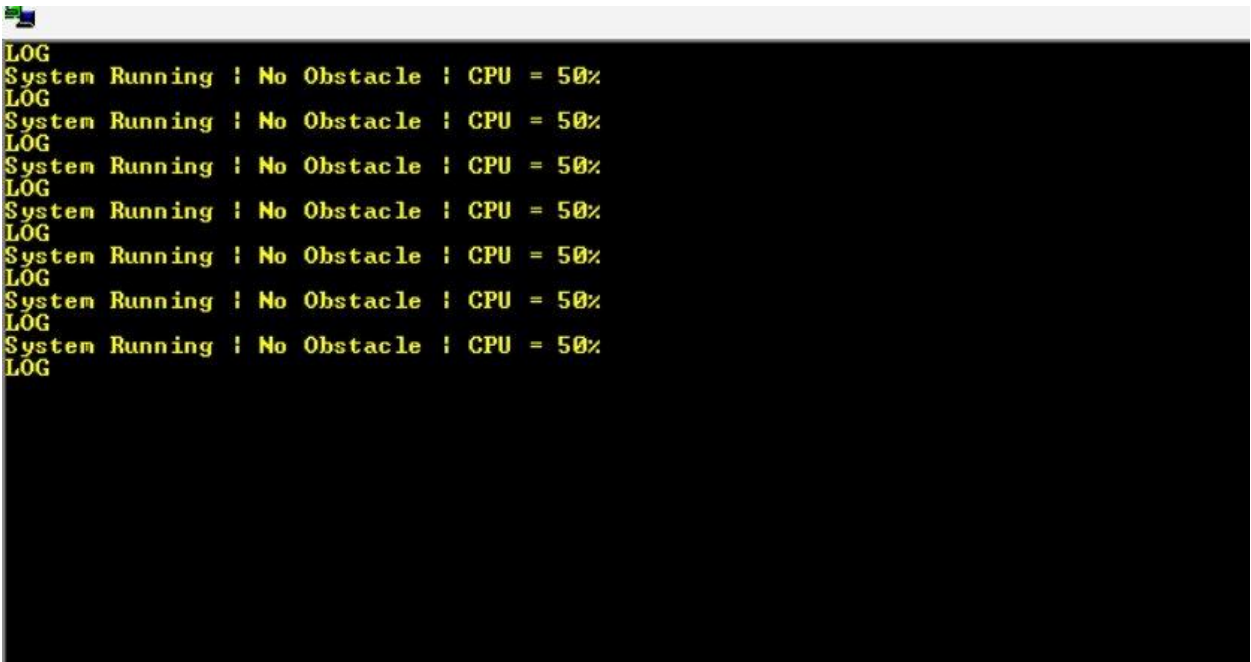


Figure 1: Normal Operation — No Obstacle Detected (CPU = 50%)

```

System Running ! No Obstacle ! CPU = 50%
stress thread
Stress Task Started
LOG

===== CRITICAL MODE =====
CPU = 99%
Critical Tasks Running: IR, Brake, Motor
Non-Critical Tasks Stopped: Logging, LED

```

Figure 2: Critical Mode Triggered — Stress Task Active (CPU = 99%), Non-Critical Tasks Suspended

```

LOG
System Running ! No Obstacle ! CPU = 50%
LOG
System Running ! No Obstacle ! CPU = 50%
LOG
System Running ! Obstacle Detected ! CPU = 50%
LOG
System Running ! Obstacle Detected ! CPU = 50%
LOG

```

Figure 3: Obstacle Detection — System Transitions from “No Obstacle” to “Obstacle Detected” State

6. Mixed-Criticality Analysis & Limitations

6.1 Criticality Levels

Criticality	Tasks	In Critical Mode?	Justification
HIGH	BreakTask, IRSensorTask	YES	Direct safety impact — 50 ms deadline must be met
MEDIUM	MonitorTask, MotorTask	YES (modified)	Monitoring continues; switches output method
LOW	LoggingTask, defaultTask	NO (suspended)	Non-safety function; expendable under resource pressure

6.2 Current Limitations & Suggested Improvements

- Critical mode is one-way — no recovery even if load drops below 80%. A hysteresis mechanism (resume tasks when CPU < 60%) would improve robustness.
- MotorTask and defaultTask are currently passive placeholders consuming stack without meaningful work.
- No watchdog timer — a misbehaving task could starve lower-priority tasks without recovery.
- Suggested: Add IWDG watchdog, interrupt-driven IR detection (sub-ms response), DMA-based UART, and PWM motor/buzzer control.

7. RTOS Concepts Applied in This Project

This section maps the core RTOS and embedded systems concepts covered in theory to their concrete implementation in this project.

RTOS Concept	Implementation in Project	How It Is Used
Preemptive Scheduling	FreeRTOS Scheduler	BreakTask (High priority) preempts all lower-priority tasks instantly when the scheduler runs, ensuring 50 ms worst-case brake response regardless of other task activity.
Task Priorities (RMS)	Rate Monotonic Scheduling	Six tasks are assigned priorities following RMS — shorter-period, safety-critical tasks (BreakTask, IRSensorTask at 50 ms) receive higher priorities than slower tasks (MonitorTask, LoggingTask at 1000 ms).
Inter-Task Communication	Event Flags (osEventFlags)	IRSensorTask sets/clears OBSTACLE_BIT via osEventFlagsSet(). BreakTask reads this flag non-blocking to decide motor/buzzer state — a clean producer-consumer IPC pattern without shared variables.
Mutual Exclusion	Mutex (osMutex)	uartMutex serializes all printf() calls across tasks using osMutexAcquire/Release with osWaitForever, preventing garbled UART output when multiple tasks print simultaneously.
Dynamic Task Management	osThreadSuspend / osThreadNew	In Critical Mode, MonitorTask suspends LoggingTask at runtime via osThreadSuspend(). StressTask is created dynamically at runtime via osThreadNew() on button press, demonstrating runtime task lifecycle management.
ISR-to-Task Communication	Volatile Flag / EXTI ISR	The PC13 button EXTI ISR sets a volatile stress_request flag. MonitorTask polls this flag in its task loop and spawns StressTask accordingly — a safe ISR-to-task signalling pattern.
CPU Load Monitoring	Idle Hook + DWT Cycle Counter	MonitorTask uses the FreeRTOS Idle Hook alongside the ARM DWT CYCCNT cycle counter. A 2-second calibration baseline is captured at startup; subsequent 1-second windows compute $CPU\% = 100 - (idle/max_idle) \times 100$.
Mixed-Criticality Scheduling	Adaptive Critical Mode	When CPU load exceeds 80%, the system autonomously transitions to Critical Mode — shedding LOW criticality tasks (Logging, LED) to protect HIGH criticality deadlines (IR sensing, braking). This mirrors the Vestal mixed-criticality model.
Deterministic Timing	osDelay() / Periodic Tasks	Each task uses osDelay() to yield CPU and enforce its period precisely. Safety tasks run every 50 ms, monitoring tasks every 1000 ms, achieving deterministic periodic execution guaranteed by the RTOS tick.

8. Conclusion

This project successfully demonstrates a functional Mixed-Criticality Real-Time System on the STM32F4 Nucleo using FreeRTOS. The system correctly implements priority-based preemptive scheduling, event-flag inter-task communication, mutex-protected shared resource access, software CPU load measurement, and adaptive task management through dynamic task suspension.

Terminal output confirms all three operational scenarios — normal operation, obstacle detection, and overload/critical mode — behave as designed. The 50 ms brake response guarantee is maintained even under 99% CPU load, validating the core promise of RTOS-based real-time design. This system serves as a strong foundation for more sophisticated automotive safety systems and directly mirrors techniques used in production automotive ECUs operating under ISO 26262 safety standards.

References

- STMicroelectronics. STM32F4 Reference Manual (RM0090). Rev 20, 2022.
- FreeRTOS. FreeRTOS Reference Manual v10.0.0. Real Time Engineers Ltd., 2018.
- Vestal, S. Preemptive Scheduling of Multi-Criticality Systems. RTSS 2007.
- Burns, A. & Davis, R. Mixed Criticality Systems — A Review. University of York, 2017.
- Barry, R. Mastering the FreeRTOS Real Time Kernel. Real Time Engineers Ltd., 2016.